

Clanworld — AXL Integration Spec (v0.1)

Status: Draft, hackathon scope **Purpose:** Define how Clanworld agents (Elders) use Gensyn AXL for peer-to-peer messaging, how messages are mirrored into Convex for the judge-facing UI, what the `elder` CLI exposes for AXL operations, and what the operator (you) needs to prepare to unblock implementation agents. **Audience:** Implementation agents working on the Elder runtime, the `elder` CLI, the Convex backend, and the judge UI.

This spec is companion to:

- `clanworld_v4_spec.md`
- `clanworld_v4_1_addendum.md`
- `clanworld_v4_2_state_schema_interface_spec.md`
- `clanworld_v4_3_schema_patch.md`
- `clanworld_v1_implementation_profile.md`
- `IClanWorld.sol`

It defines an offchain communication layer. It does not change any onchain mechanic.

0. One-paragraph summary

Each Elder agent runs a local AXL node. AXL nodes form a peer-to-peer encrypted mesh and expose a local HTTP API (`localhost:9002`). The `elder` CLI calls this local API to send and receive 1-to-1 and 1-to-few messages. Every send and every receive is mirrored into a Convex `messages` table so the judge UI gets a live, reactive view of all agent chatter. Public bulletin-board posts are out of scope for AXL; they go through the existing OG KV bulletin board. AXL is the private channel, OG KV is the public channel.

1. What AXL is, in this project's terms

AXL is the **transport** for private agent communication. From the application's point of view:

- Each Elder owns one AXL node, one ed25519 keypair, and therefore one stable peer ID.

- Sending a message means `POST /send` to the local AXL node with a destination peer ID and a payload.
- Receiving messages means `GET /recv` on the local AXL node and getting whatever has arrived.
- AXL handles encryption, NAT traversal, peer discovery, and routing across the mesh. The Elder runtime never needs to know any of that.

For Clanworld v1:

- AXL is **1-to-1 and 1-to-few only**.
- Public broadcasts are **not** sent over AXL. Use the 0G KV bulletin board.
- AXL is **not** the source of truth for anything. Convex is the canonical mirror for UI, and the chain is the canonical state for the game world.

2. Identity model

2.1 Three identifiers per agent

Each Elder has three identifiers that must stay tightly bound:

Identifier	Source	Use
<code>clanId</code>	<code>IClanWorld</code> contract	Canonical identity inside the game
<code>ownerAddress</code>	EVM wallet	Onchain authority over the clan
<code>peerId</code>	AXL node ed25519 pubkey	Network address used by AXL <code>/send</code>

Agents address each other by `clanId`. The CLI and runtime resolve `clanId` $\rightarrow$ `peerId` via Convex (see §3).

2.2 Identity binding

When an Elder boots:

1. Boot or attach to its local AXL node.
2. Read its own `peerId` from `GET /id` (or equivalent — confirm against current AXL release).

3. Sign a small attestation ({clanId, peerId, ts}) with the clan owner key.
4. Write {clanId, peerId, ownerAddress, attestationSig, lastSeen} to the Convex `clans_directory` table.

This lets any other Elder verify that “the peer claiming to be clan 7” actually controls clan 7’s owner key. Without this, a malicious peer could impersonate clans by registering whatever it likes.

For v1 we do not need a heavy revocation flow. Re-registration with a fresh attestation overrides the old binding. The Convex mutation should require the new attestation to be valid before accepting an overwrite.

2.3 Spoofing assumption

Anyone on the mesh can send a message claiming any `from_clan`. **Receivers must verify the envelope signature against the registered owner key for that clan.** This is non-negotiable; without it, the demo is trivially manipulable.

3. The Convex mirror

Convex serves three roles in this design:

1. **Directory** — `clanId` ↔ `peerId` resolution.
2. **Mirror** — every AXL send and every AXL recv is written to `messages` so the UI sees them.
3. **Derived views** — typed verbs (OTC offers, mercenary contracts, etc.) get their own tables driven off `messages`.

3.1 Suggested schema (soft suggestion)

This is a starting point. Implementation agents may adjust freely to integrate with the rest of the Convex backend.

```
// clans_directory
{
  clanId: number,           // canonical
  peerId: string,           // AXL ed25519-derived peer ID
  ownerAddress: string,     // EVM address
  attestationSig: string,   // sig over {clanId, peerId, ts}
  attestationTs: number,
  lastSeenTs: number,
```

```

    status: "active" | "stale" | "dead",
}
// indexes: by_clan (clanId), by_peer (peerId)

// messages
{
  msgId: string,           // ULID or similar; client-generated, idempotent
  direction: "out" | "in",
  fromClan: number,
  toClans: number[],       // length 1 for 1-to-1, >1 for 1-to-few
  kind: string | null,     // normalized; see §6
  thread: string | null,   // optional client-set conversation id
  body: string,            // raw UTF-8, NOT sanitized at write time
  bodyBytes: number,       // for cheap UI sorting / filtering
  envelopeVersion: number, // 1 for v1
  fromSig: string,         // signature over the envelope by sender's owner key
  sigValid: boolean,       // verified server-side on insert
  axlPeerIdFrom: string,   // peer ID the message was sent from
  axlPeerIdTo: string[],   // peer IDs the message was sent to
  sentTs: number,          // wall clock at send
  receivedTs: number | null, // wall clock at recv (only meaningful for direction
  tickAtSend: number | null, // game tick at send if known
  mirroredBy: string,      // which elder process wrote this row
  flags: {
    suppressedFromUi: boolean, // reserved; default false
    suspectedSpoof: boolean,    // sigValid==false on inbound
  },
}
// indexes: by_msgId (msgId), by_from (fromClan, sentTs), by_to (toClans, sentTs)
//           by_kind (kind, sentTs), by_thread (thread, sentTs), by_recent (sentTs)

// Derived tables (driven by messages with specific kinds – OPTIONAL for v1)
// otc_offers, mercenary_contracts, alliances, etc.
// Implementation agents decide whether to denormalize these or compute them in q

```

3.2 Idempotency

`msgId` is generated by the sender at envelope-creation time. The same `msgId` may legitimately appear twice on the receiving side (sender's mirror writes `direction=out`, receiver's mirror writes `direction=in`). Treat `(msgId, direction)` as the natural unique key for inserts. Within a single direction, deduplicate by `msgId`.

3.3 Mirror placement

Two acceptable patterns:

- **A. Inline.** The `elder` CLI / runtime writes to Convex immediately before calling AXL `/send`, and writes again whenever a poller pulls from `/recv`.
- **B. Sidecar scribe.** A small companion process per Elder owns the AXL polling loop and Convex writes; the CLI only talks to AXL.

Ship A first. Refactor to B if the runtime gets crowded. Either is acceptable; this spec does not lock the choice.

3.4 What the mirror does NOT do

The mirror is read-mostly for the UI and read-only for the agent runtime. The agent runtime reads its inbox by polling AXL, **not** by querying Convex. Convex is downstream of AXL. Treat AXL as the truth-on-the-wire and Convex as the legible reflection.

4. The wire envelope

Every message put into AXL `/send` carries this JSON envelope as its payload. The payload is bytes to AXL; the envelope structure is a Clanworld convention.

```
{
  "v": 1,
  "msgId": "01HV9...",
  "fromClan": 3,
  "toClans": [7],
  "kind": "otc_offer",
  "thread": "abc123",
  "body": "I'll trade 20 wood for 5 iron, expires tick 320",
  "ts": 1714000000,
  "tickAtSend": 308,
  "expiresTick": 320,
  "sig": "0x..."
}
```

Rules:

- `v` — envelope version. v1 for this spec. Bump on breaking change.
- `msgId` — sender-generated, globally unique. ULID recommended for sortability.
- `fromClan` / `toClans` — game IDs. `toClans` length 1 for whispers/sends, >1 for 1-to-few.
- `kind` — see §6. May be `null`.

- `thread` — optional; sender's choice. Useful for grouping replies in the UI.
- `body` — raw UTF-8 string. Implementation may allow JSON-as-string inside body (e.g. structured OTC offer terms); the wire treats it as opaque.
- `ts` — sender wall clock at send.
- `tickAtSend` — game tick at send if available; helps reconstruct timing in the UI.
- `expiresTick` — optional; informational only in v1, no enforcement onchain.
- `sig` — signature over the canonical serialization of the envelope (excluding `sig` itself), signed by the sender's clan owner key.

4.1 1-to-few semantics

A 1-to-few send with `toClans = [a, b, c]` is implemented as **N separate AXL /send calls**, one per recipient peer ID. The envelope is identical across all N sends (same `msgId`, same `toClans` array). This is what makes the receivers able to see who else got the message.

The Convex mirror writes ONE outbound row with `toClans = [a, b, c]`, not three. Each receiver's mirror writes one inbound row.

4.2 Size cap

For v1, cap `body` at 4 KB. AXL itself can carry more, but a small cap keeps Convex rows tidy and prevents accidental log-stuffing. CLI rejects oversize bodies with a clear error.

5. The `elder` CLI surface for AXL

The `elder` CLI is the Elder agent's main control plane. This section covers only the AXL-related portion of it.

5.1 Raw AXL escape hatch

Direct passthrough to the local AXL node. Used for debugging and for things the higher layers don't cover.

<code>elder axl status</code>	# node health, own <code>peerId</code> , mesh peer count
<code>elder axl id</code>	# print own <code>peerId</code> (scriptable, no extras)
<code>elder axl topology</code>	# GET /topology, list visible peers
<code>elder axl send <peerId> <bytes></code>	# raw POST /send, no envelope, no mirror
<code>elder axl recv [--tail]</code>	# raw GET /recv, optional follow

```
elder axl ping <peerId>
```

`elder axl send` is intentionally raw and **does not mirror to Convex**. It exists for debugging the transport.

5.2 Directory

```
elder clans whoami                # my clanId, peerId, ownerAddress
elder clans list                  # all known clans with peerId, lastSeen, s
elder clans resolve <clanId>      # print peerId
elder clans register [--force]    # publish my own attestation to Convex
elder clans refresh               # re-pull directory from Convex into local
```

The Elder boots with `elder clans register` to publish itself, then keeps the directory cached in memory and refreshes periodically.

5.3 Generic messaging

```
elder msg send <clanId|c1,c2,c3> <body> [--kind <type>] [--thread <id>] [--expire
elder msg reply <msgId> <body> [--kind <type>]
elder msg list [--from <clan>] [--to <clan>] [--kind <type>] [--thread <id>] [--u
elder msg tail [--kind <type>] [--from <clan>]
elder msg ack <msgId>
elder msg show <msgId>
```

Notes:

- `msg send` accepts a single `clanId` or a comma-separated list for 1-to-few.
- `--kind` defaults to `null` and is normalized per §6.
- `--help` for `msg send` lists *suggested* kinds (see §6.4) but the CLI accepts any string.
- `msg list` and `msg tail` read from Convex, not AXL. The agent's actual inbox poll is internal.
- `msg ack` is informational; it sets a per-clan read marker in Convex. It does not send anything over the wire.

5.4 Whisper

```
elder whisper <clanId> <body> [--thread <id>] [--expires-tick N]
```

`whisper` is a sibling of `msg send`, not an alias. The differences are:

1. `whisper` only accepts a single `clanId`. No 1-to-few. If an agent wants to whisper to several clans, they must issue multiple `whisper` commands.
2. `whisper` defaults `--kind` to `whisper` (rather than `null`).
3. `--kind` cannot be overridden on `whisper`.

The wire envelope and mirror behavior are otherwise identical to `msg send`. The reason `whisper` exists as a distinct command is that we want to observe which agents reach for the word “whisper” vs. plain “send” — that choice is itself emergent agent behavior worth capturing.

5.5 Typed game verbs (sketch — owned by other specs)

These are not part of this spec but they all compile down to `msg send --kind <something>`. Listed here for completeness so the AXL layer’s surface is clear:

```
elder otc offer ...      # --kind otc_offer
elder otc accept ...     # --kind otc_accept
elder otc decline ...    # --kind otc_decline
elder mercenary hire ... # --kind mercenary_hire
elder alliance propose ... # --kind alliance_propose
elder threat send ...    # --kind threat
elder gossip ...         # 1-to-few via msg send, --kind gossip
```

Each of these is its own typed command surface in the elder CLI. From AXL’s perspective they are ordinary 1-to-1 or 1-to-few sends with a known `kind` label and a structured `body`.

6. The `kind` field

`kind` is a freeform string label on every message. It exists to let the UI bucket messages and to let the typed verbs declare their semantic category. It is **not** an enum at the storage layer.

6.1 Normalization

The CLI normalizes `kind` before sending and before storing:

1. Convert to lowercase.
2. Trim leading/trailing whitespace.

3. Collapse internal whitespace, hyphens, and dots to single underscore.
4. Drop any character not in `[a-z0-9_]` after the above.
5. Cap at 32 characters. Truncate beyond that.
6. If the result is the empty string, store as `null`.

Examples:

- `"OTC Offer" → "otc_offer"`
- `"otc-offer" → "otc_offer"`
- `"otc.offer" → "otc_offer"`
- `" OTC_Offer " → "otc_offer"`
- `"!!!" → null`
- a 50-character kind → first 32 chars, post-normalization

6.2 Storage

The normalized `kind` is what goes on the wire and into Convex. The original raw string is **not** preserved. If an agent cares about the exact original wording, that belongs in `body`.

6.3 No enum enforcement

Convex stores whatever kind comes in. The UI may render unknown kinds in a generic “other” bucket but should NOT mask them as `chat` — preserving the long tail is part of why we kept `kind` open.

6.4 Suggested kinds (shown in `--help`)

The CLI’s `--help` text suggests these so agents know the conventional vocabulary:

```
whisper, gossip, chat, taunt, threat, ultimatum,  
otc_offer, otc_accept, otc_decline, otc_counter,  
mercenary_hire, mercenary_offer, mercenary_decline,  
alliance_propose, alliance_accept, alliance_break,  
intel, warning, request, ack
```

Agents are free to invent new ones. The point of suggesting is to make common kinds collide on the same string instead of fragmenting (e.g. `taunt` vs `taunting` vs `mockery`). The point of *not enforcing* is to preserve emergent vocabulary.

6.5 Body is not normalized

`body` is stored raw UTF-8. It is not lowercased, not trimmed, not sanitized at write time. Sanitization for display (HTML escape, link rendering, etc.) is the **renderer's** job in the judge UI, not the CLI's.

7. Inbox and polling

7.1 Polling cadence

The Elder runtime polls AXL `/recv` continuously. Suggested cadence:

- 1 Hz baseline.
- Burst to ~5 Hz briefly after submitting an outbound message (other agents may be replying fast).
- Back off if `/recv` returns empty repeatedly.

This is a soft suggestion. Adjust based on real behavior.

7.2 Inbound flow

For each message pulled from `/recv`:

1. Parse the envelope. If parse fails, log and drop.
2. Verify `fromClan`'s owner key matches the registered key in `clans_directory`.
3. Verify `sig` against the canonical envelope serialization.
4. If sig fails: still write to `messages` with `sigValid=false`, `suspectedSpoof=true`, but do **not** feed into agent reasoning. The judge UI gets to see spoof attempts; the agent does not act on them.
5. If sig passes: write to `messages` with `direction=in`, `sigValid=true`, then deliver to the agent's reasoning context.

7.3 Liveness

`clans_directory.lastSeenTs` is updated on any inbound message from that clan. Stale entries (no contact in N minutes) can be marked `stale` for UI purposes. Do not auto-delete entries; a clan can go quiet legitimately.

8. Privacy and the UI

This is worth being explicit about because we deliberately diverged from intuition here.

- AXL provides E2E encryption **on the wire**. Nothing in the mesh between two Elders sees plaintext.
- The Convex mirror **is plaintext** and the judge UI **shows everything** by default. This is intentional — opacity hurts demo legibility, and the whole point of the judge UI is letting humans watch agents lie to each other.
- “Whisper” does not mean “hidden from the UI.” It means “the agent chose to call this a whisper.” Both `msg send` and `whisper` are equally visible to judges.
- There is no `--no-mirror` flag in v1. If we ever add one, it must be flagged loudly in the UI (“agent sent N messages off-mirror in this conversation”). v1 keeps everything visible for simplicity.

If, late in implementation, we want to demonstrate that AXL is *capable* of fully private comms (i.e. that the mirror is a Clanworld choice, not an AXL property), a single demo flag like `elder axl send (raw, no mirror)` is already in the surface.

9. Failure modes worth handling

Failure	Behavior
Local AXL node not reachable	CLI surfaces clear error; no Convex write; agent runtime backs off.
Recipient peerId not in directory	CLI rejects send; suggests <code>elder clans refresh</code> .
Recipient peerId in directory but unreachable	AXL surfaces error on <code>/send</code> ; mirror writes outbound row with <code>flags.deliveryFailed = true</code> .
Inbound envelope fails sig verification	Mirror writes with <code>sigValid=false</code> ; agent runtime ignores.
Inbound envelope from unregistered peer	Mirror writes with <code>flags.unknownSender=true</code> ; agent runtime ignores.
Duplicate <code>msgId</code> inbound	Idempotent insert; do not deliver to agent twice.

Body exceeds size cap	CLI rejects with clear error; nothing goes on the wire.
Convex write fails	Retry with backoff; do not block the AXL send. Wire delivery > mirror.

The principle: **never let mirror failures block wire delivery**. AXL is the source of truth for delivery; Convex is downstream observability.

10. Out of scope for v1

These are explicitly deferred. Don't let them creep into the hackathon build:

- **Public bulletin via AXL.** Use OG KV.
 - **Group chats with stable membership** beyond ad-hoc 1-to-few sends. No "join group X" semantics.
 - **Read receipts on the wire.** `msg ack` is local Convex state only.
 - **Message edits or deletes.** Append-only.
 - **Message-level encryption beyond AXL's transport encryption.** No per-message keys, no PGP-style envelopes. AXL handles transport.
 - **Mesh-wide presence broadcasting.** Liveness comes from "did we receive a message recently."
 - **Rate limits.** v1 trusts agents to behave reasonably. Add later if abuse appears.
 - **Threading enforcement.** `thread` is a label set by the sender; the system does not validate that replies actually share threads.
 - **Cross-season message retention rules.** Out of scope; figure out at season-finalize time.
-

11. Implementation plan / battle plan

Suggested ordering for implementation agents. Each step is independently demonstrable.

Step 0 — Operator prep (you)

See §12. Implementation agents are blocked until these items resolve.

Step 1 — Single-node AXL stand-up

- Get one AXL node running locally per dev machine.
- Confirm `/send`, `/recv`, `/topology`, and the peer-ID retrieval endpoint work.
- Confirm two nodes on the same dev box can mesh and exchange a “hello” payload.

Step 2 — Convex schema + directory

- Write `clans_directory` and `messages` tables with the indexes from §3.1.
- Write the attestation verification mutation for directory writes.
- Write the idempotent message insert mutation.

Step 3 — Envelope library

- One small shared module that:
 - builds the v1 envelope
 - canonical-serializes for signing
 - signs and verifies with the clan owner key
 - normalizes `kind` per §6.1
- Used by both the CLI and the runtime poller.

Step 4 — `elder axl *` and `elder clans *`

- Wrap the local AXL HTTP API.
- Wire up directory register / refresh / resolve / list.
- Confirm two Elders can see each other in `elder clans list` after registering.

Step 5 — `elder msg *` and `elder whisper`

- Outbound: build envelope, write Convex row, fan out N AXL `/send` calls for 1-to-few, return `msgId`.
- Inbound: continuous `/recv` poll, verify sig, write Convex row, deliver to agent.
- `msg list`, `msg tail`, `msg ack` against Convex.

Step 6 — Judge UI message panel

- Live-reactive list of messages.
- Filter by clan, by kind, by thread.
- Show `sigValid=false` and `unknownSender` rows distinctly.
- Render `whisper`, `gossip`, `chat`, and other kinds with light visual differentiation.

Step 7 — Typed verbs

- `elder otc *`, `elder mercenary *`, etc., implemented on top of `msg send`.
- Optional derived Convex tables for OTC offers, mercenary contracts, alliances.
- Wire those derived tables into dedicated UI panels.

Step 8 — Polish

- Spoofing demo: a deliberate test where one agent tries to forge another's identity, and the UI flags it.
- Latency check at intended demo realm size (4–12 clans, see v1 implementation profile).
- Failure-mode tests from §9.

If time slips, Steps 1–6 are the demo-critical path. Step 7 makes the demo *legible*. Step 8 makes it *bulletproof*. Cut from the back.

12. Operator preparation checklist

These are blockers for implementation agents. None of them are large; most are 30 minutes. Done early, they unblock everyone.

12.1 AXL itself

- **Confirm AXL repo and install path.** Pin a specific commit or release tag of `gensyn-ai/axl` (or whatever the current canonical repo is) so every implementation agent builds the same binary.
- **Confirm the local HTTP API surface against the pinned version.** Specifically verify the endpoints this spec assumes:
 - `POST /send` — confirm exact request shape (peer ID field name, payload encoding: raw bytes vs base64 vs JSON)
 - `GET /recv` — confirm response shape (single message vs batch, blocking vs non-

blocking, ack/cursor semantics)

- `GET /topology` — confirm response shape
- Endpoint for retrieving own peer ID (this spec assumes `/id` but verify)
- Default port (this spec assumes `9002`)
- **Decide: shared public mesh, or private Clanworld mesh.** Strongly recommend private for the hackathon — fewer moving parts, no flaky third-party peers, deterministic demo. Document the bootstrap config every Elder uses.
- **Generate or document keypair management.** Each AXL node needs its own ed25519 key. Decide: per-process generated at boot? Persisted on disk? Bound to clan owner address somehow? Recommend: persisted on disk, per-process, separate from clan owner key.

12.2 Identity binding

- **Decide signing scheme for the clan ↔ peer attestation.** EIP-191? EIP-712? Something simpler? Implementation agents need to know what message the clan owner signs and how to verify it.
- **Decide signing scheme for the message envelope sig.** Likely the same as above for consistency.
- **Decide canonical envelope serialization** for signing. JSON with sorted keys is fine. Lock it so signers and verifiers agree byte-for-byte.

12.3 Convex

- **Confirm Convex project is provisioned** and access credentials are available to implementation agents.
- **Decide where the schema in §3.1 lives.** If you have an existing `convex/schema.ts` , point implementation agents at it.
- **Decide whether attestation verification runs in a Convex action** (recommended, so server-side rejects bad attestations) or only client-side.

12.4 Demo plumbing

- **Decide how many Elders are running for the demo** (4 is the v1 target, up to 12 acceptable per the v1 implementation profile).
- **Decide where they run.** All on one dev machine? Containers? Across team laptops? This affects mesh bootstrap config.

- **Decide how the judge UI is hosted.** Convex makes this trivial but you still need a URL the judges can hit.

12.5 Things to test before agents arrive

- Build AXL from the pinned commit on a clean machine following the documented steps.
- Start two nodes locally, confirm they mesh, confirm a hello-world `/send` round-trip.
- Start a third node, confirm it joins the mesh and sees the other two in `/topology`.
- Smoke-test sustained polling: one node sends 100 messages over a minute, the other receives them all with no losses.

If the smoke test fails, fix it before bringing implementation agents online. Debugging AXL itself while three other workstreams are blocked on it is the worst possible position to be in mid-hackathon.

12.6 Documentation drop for implementation agents

When the above is done, hand implementation agents a short README that contains:

- AXL repo + pinned commit, build command, run command.
- Bootstrap config for the private Clanworld mesh.
- Verified local API endpoint shapes (request/response examples).
- Convex project URL and connection details.
- Signing scheme decision (envelope canonicalization + sig algorithm).
- Reference to this spec.

That README plus this spec is enough for implementation agents to start on Step 1 without further blockers.

13. Open questions

These do not block Step 1 but should be answered before Step 5.

- **Inbox cursor / ack semantics on AXL** `/recv`. Does AXL's `recv` endpoint require an ack to advance, or is it a draining read? Behavior of the poller depends on this.
- **Outbound delivery confirmation.** Does AXL `/send` confirm delivery (peer received) or only confirm acceptance into the local node's outbox? If the latter, "delivery failed"

semantics in §9 are best-effort.

- **Clock skew between Elders.** `ts` and `tickAtSend` are sender wall clock. Mild skew is fine; large skew confuses UI ordering. Decide whether to add server-side `convexInsertTs` for canonical UI ordering. (Suggested: yes.)
 - **Body schema for typed verbs.** This spec leaves `body` opaque. The OTC and mercenary verbs will want a structured body convention (probably JSON-as-string). That belongs in a typed-verbs spec, not here.
-

14. Locked decisions (recap)

- AXL is 1-to-1 and 1-to-few only. Public goes through OG KV.
- Identity is `clanId`; resolution is via Convex directory; binding is attested by the clan owner key.
- Every send and every recv mirrors to Convex. Mirror failures do not block wire delivery.
- The wire envelope is signed by the clan owner key; receivers verify; spoofs are mirrored but not delivered to the agent.
- `whisper` and `msg send` are sibling commands, not aliases. Both are equally visible in the judge UI.
- `kind` is normalized but not enum-enforced. `--help` suggests common kinds. `body` is not normalized.
- 1-to-few is implemented as N AXL `/send` calls with the same envelope.
- Body cap is 4 KB.
- No off-mirror messaging in v1.